UNIT 2

SEARCHING:

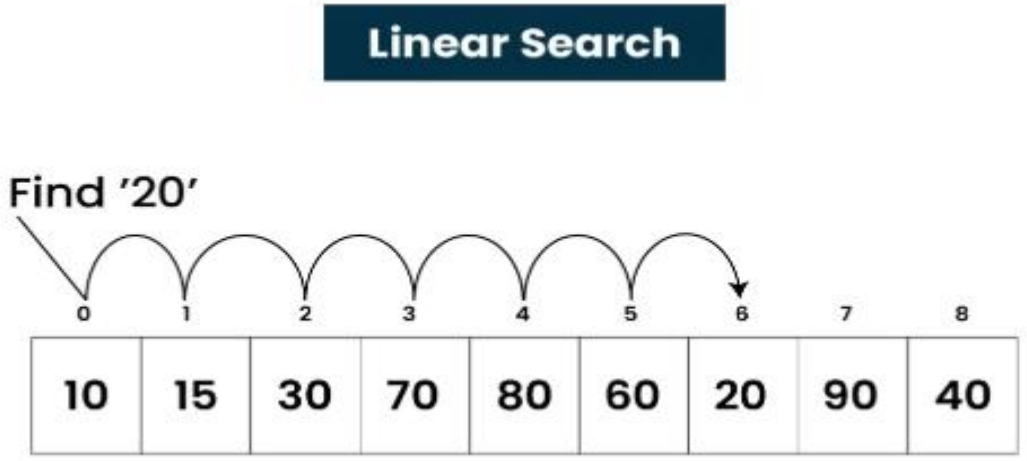
- **Searching** refers to the operation of finding the location of a given item in a collection of items.
- 2 types of searching techniques are

  a)Linear search/Sequential search

  b)Binary search

Linear /Sequential search

- It is one of the simplest and most straightforward searching algorithms.
- It works by sequentially examining each element in a collection of data(array or list) until a match is found or the entire collection has been traversed.



ALGORITHM FOR LINEAR SEARCH

Linear Search(A, Item)
Step 1: [Initialize] Set i = 0
Step 2: Repeat while i < n:
Step 3: If A[i] == Item, then:
        Print “Item found at index i"
        Exit
Step 4: Increment i = i + 1
Step 5: Print “Item not found"
Step 6: Exit

Binary Search

- Binary Search is an efficient searching algorithm used to find an element in a sorted array.
- Instead of checking elements one by one (like Linear Search), it divides the search space into halves and eliminates half of the elements in each step.

Working Principle:

- Compare the target value with the middle element of the array.
- If they are equal, return the index of the middle element.
- If the target value is smaller than the middle element, search the left half of the array.
- If the target value is larger than the middle element, search the right half of the array.
- Repeat this process until the element is found or the search space becomes empty.

ALGORITHM FOR BINARY SEARCH

BINARY(DATA, LB, UB, ITEM,LOC)

1. Set BEG := LB, END := UB and MID = INT((BEG + END)/2)

2. Repeat steps 3 and 4 while BEG <= END and DATA[MID]!=ITEM

3. If ITEM < DATA[MID], then:

Set END := MID – 1

else:Set BEG := MID + 1

4. Set MID := INT((BEG + END)/ 2)

5. If DATA[MID] = ITEM,

then:set LOC := MID

Else

Set LOC := NULL

6. Exit

Example:

0 1 2 3 4 5 6

DATA = [2, 4, 6, 8, 10, 12, 14]
ITEM = 10
LB = 0
UB = 6

1.Initialize:

BEG = 0, END = 6
MID = INT((0+6)/2) = 3
DATA[MID] = DATA[3] = 8

2.Compare ITEM and DATA[MID]:
ITEM = 10 > 8 → search right half
BEG := MID + 1 = 4

3.Update MID:
MID = INT((4 + 6)/2) = INT(10/2) = 5
DATA[MID] = DATA[5] = 12

4.Compare ITEM and DATA[MID]:
ITEM = 10 < 12 → search left half
END := MID - 1 = 4

5.Update MID:
MID = INT((4 + 4)/2) = 4
DATA[MID] = DATA[4] = 10

6.Check final position:
DATA[MID] = ITEM → 10 = 10
LOC := MID = 4
Item found at index 4.

Memory Allocation and De-allocation in C

- Memory allocation is the process of reserving sections of memory in a program to store variables, instances of structures, and objects. There are two types of memory allocation in C:

1.Static Allocation (Compile-time) – Memory is allocated at the time of compilation and remains fixed throughout the program execution.

2.Dynamic Allocation (Run-time) – Memory is allocated during program execution using pointers.

Dynamic Memory Allocation Functions in C

- C provides four standard library functions for dynamic memory management:These function comes under stdlib.h header file
- malloc( )
- calloc( )
- free( )
- realloc( )

1. malloc( ) Function

- The malloc() function (Memory Allocation) is used to allocate a block of memory in bytes. The function returns a pointer to the allocated memory. The allocated memory contains garbage values by default.

Syntax:

- ptr = (data_type*) malloc(number_of_elements * size_of_each_element);

Example:

int *ptr;

ptr = (int *) malloc(10 * sizeof(int));

- Here, malloc() allocates memory for 10 integers dynamically, and the pointer ptr stores the address of the allocated memory.

2. calloc( ) Function

- The calloc() function (Contiguous Allocation) works similarly to malloc() but initializes the allocated memory to zero. It requires two arguments: the number of elements and the size of each element.

Syntax:

ptr=(data_type*)calloc(number_of_elements,size_of_each_element;

Example:

int *ptr;

ptr = (int *) calloc(10, sizeof(int));

- Here, calloc() allocates memory for 10 integers and initializes all memory blocks to zero.

3. free( ) Function

- The free() function is used to deallocate memory that was previously allocated using malloc() or calloc(). It prevents memory leaks by freeing up unused memory.

Syntax:

- free(ptr_var);

Example:

- free(ptr);
- This releases the allocated memory, making it available for future use.

4. realloc( ) Function

- The realloc() function is used to resize an already allocated memory block. It is useful in situations where:
- The originally allocated memory is insufficient.
- More memory was allocated than required.

Syntax:

ptr_var = (data_type*) realloc(ptr_var, new_size);

Example:

ptr = (int *) realloc(ptr, 20 * sizeof(int));

- This reallocates memory to store 20 integers instead of the originally allocated 10 integers.

CHAPTER 3: LINKED LISTS

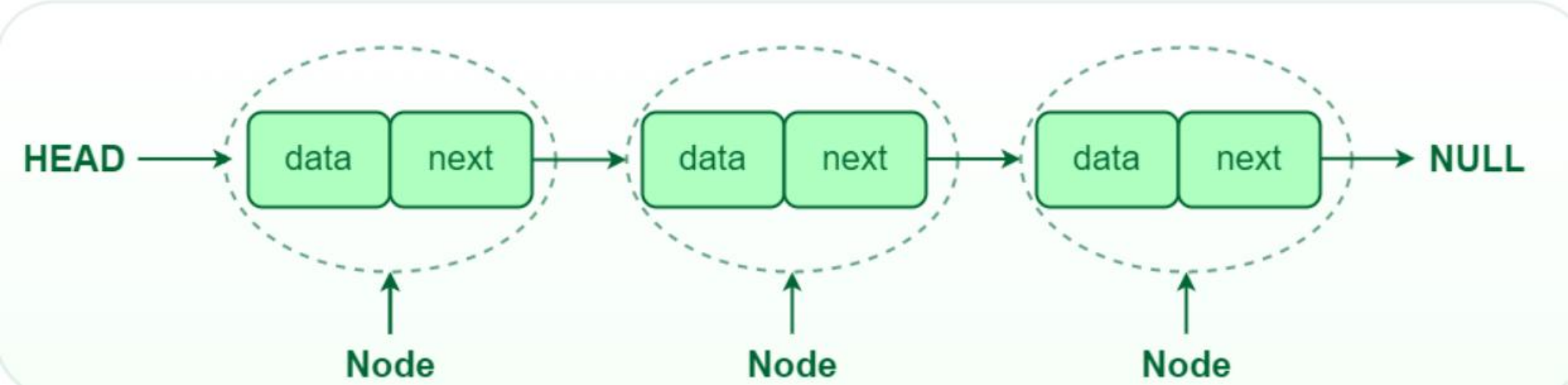
A **linked list** is a **linear data structure** consisting of a **collection of nodes**. Each **node** contains:

1. **Data field**: Stores the actual data (it could be an integer, array, structure, or any data type).
2. **Pointer (or Link field)**: Stores the address of the next node in the list.

Since nodes are connected using pointers, a linked list **does not use contiguous memory** like arrays but rather scattered memory locations.

Why Use Linked Lists?

- Acts as a **building block** for more complex data structures like **stacks, queues, graphs, and hash tables**.
- Supports **dynamic memory allocation**, meaning the size of the linked list can grow or shrink as needed.
- Easier **insertions and deletions** compared to arrays since shifting elements is not required.


HEAD
data
next
data
next
data
next
NULL
Node
Node
Node

- In C,we can implement a linked list using the following code:

```
struct node
{
 int data;
struct node * next;
};
```

struct node→This defines a structure named node

int data; →This is an integer variable that stores data (information) in the node.

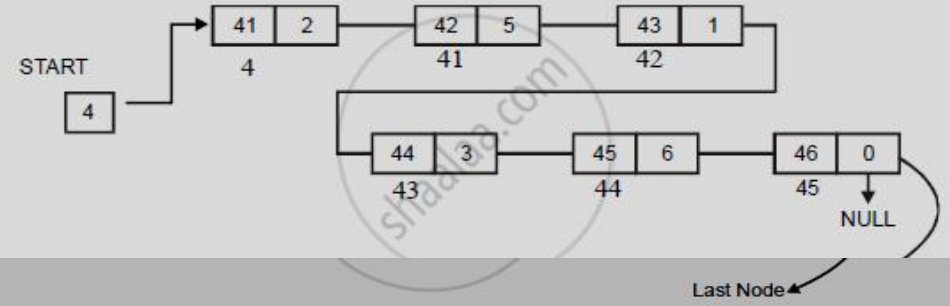
struct node* next; →This is a pointer to another structure of type node. It is used to link one node to another, forming a **linked list**.

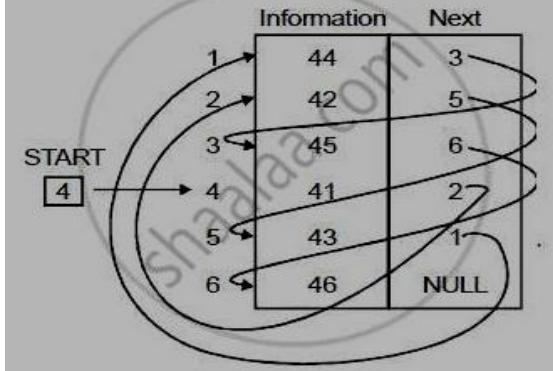
MEMORY REPRESENTATION OF LINKED LIST

(1) Linked lists can be represented in memory by using two arrays respectively known as INFO and LINK, such that INFO[K] and LINK[K] contains information of element and next node address respectively.

(2) The list also requires a variable 'Name' or 'Start', which contains address of first node. Pointer field of last node denoted by NULL which indicates the end of list. e.g., Consider a linked list given below :

(3) The linked list can be represented in memory as -





Above figure shows linked list. It indicates that the node of a list need not occupy adjacent elements in the array INFO and LINK.

Types of linked list:

- Basically, we can put linked lists into following types.

1. **Singly linked list .**
2. **Doubly linked list .**
3. **Circular linked list .**
   - **Singly circular linked list.**
   - **Doubly circular linked list.**

Singly Linked List : A **Singly Linked List (SLL)** is a linear data structure where each node contains:

1. **Data** – Stores the value.
2. **Next Pointer** – Points to the next node in the list.

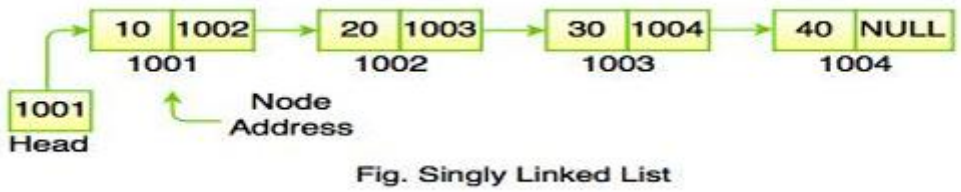


Fig. Singly Linked List

Key Features:

- The last node's next pointer is NULL , indicating the end of the list.
- Supports **unidirectional traversal** (only forward).
- Efficient **insertion and deletion** at the beginning or middle but slower at the end.

Doubly Linked List :A **Doubly Linked List (DLL)** is a linear data structure where each node contains:

1. **Data** – Stores the value.
2. **Next Pointer** – Points to the next node.
3. **Previous Pointer** – Points to the previous node.

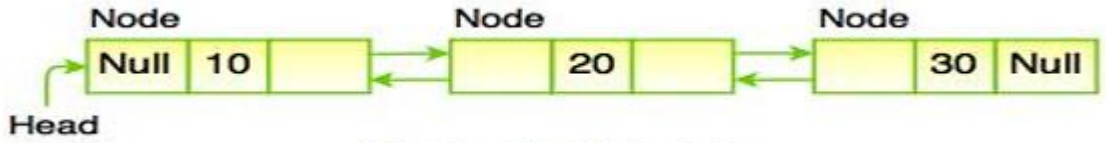


Fig. Doubly Linked List

Key Features:

- Supports **bidirectional traversal** (both forward and backward).
- The **first node's previous pointer** is NULL.
- The **last node's next pointer** is NULL.
- More efficient **insertion and deletion** than a singly linked list, especially in the middle.

- **Circular Linked List** : A **Circular Linked List (CLL)** is a variation of a linked list where the last node is connected back to the first node, forming a **closed loop**. It can be of two types:
- **Singly Circular Linked List**
- **Doubly Circular Linked List**
- **Singly Circular Linked List** – Each node has a next pointer, and the last node points back to the first node.

Key Features of Circular Linked List:

- **No NULL pointers** – The last node connects back to the first node.
- **Efficient for cyclic operations** – Ideal for applications requiring continuous traversal.
- **Can start from any node** – There is no fixed head in circular list

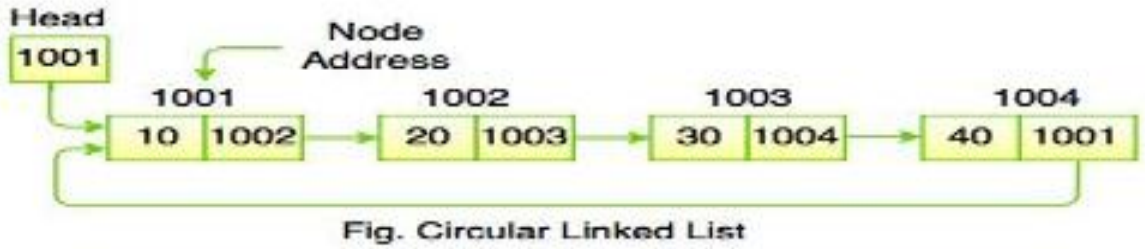


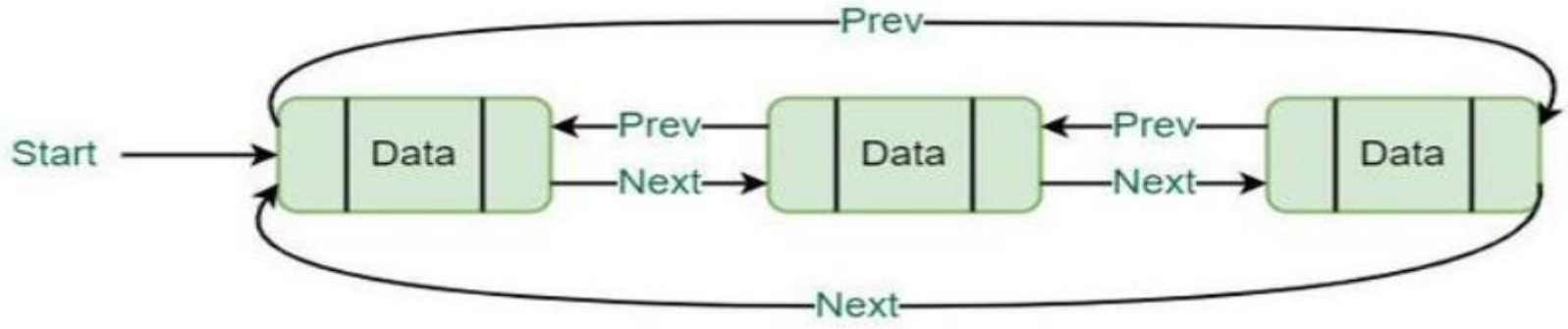
Fig. Circular Linked List

Doubly Circular Linked List : A **Doubly Circular Linked List (DCLL)** is a linked data structure where each node has three components:

1. **Data** – Stores the value.
2. **Next Pointer** – Points to the next node.
3. **Previous Pointer** – Points to the previous node.

Key Features:

- The **last node** connects to the **first node** (circular structure).
- Supports **bidirectional traversal** (both forward and backward).
- No NULL pointers, as the list forms a **closed loop**.
- Efficient **insertion and deletion** at any position.



Operations on Singly linked lists.

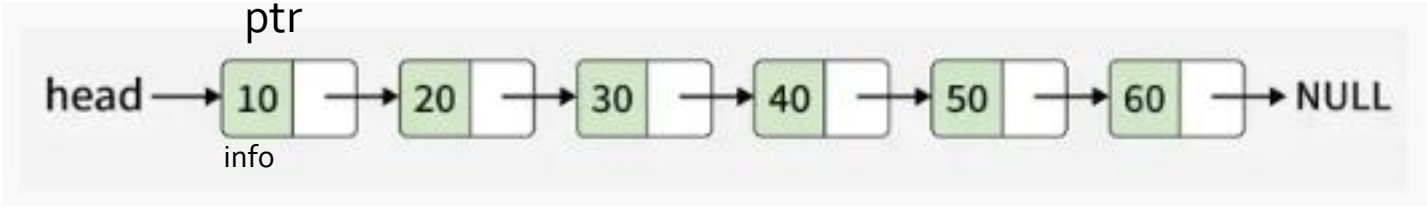
- Traversing.
- Searching.
- Insertion.
- Deletion.

Traversing a linked list

- Let LIST be a linked list in memory stored in linear arrays INFO and LINK with START pointing to the first element and NULL indicating the end of LIST.
- Suppose we want to traverse LIST in order to process each node exactly once. Algorithm uses a pointer variable PTR which points to the node that is currently being processed.
- Accordingly, LINK[PTR] points to the Next node to be processed. until PTR=NULL, Which signals the end of the list

Algorithm

1. PTR=START
2. While PTR != NULL
3. Process INFO[PTR]
4. PTR = LINK[PTR]
5. Return



Searching a linked list

- Let LIST be a linked list in memory.
- Suppose a specific ITEM of information is given.
- searching algorithms for finding the location LOC of the node where ITEM appears in LIST.
- Assume element in list are not in sorted order.
- if ITEM doesn't found SET LOC=NULL

Algorithm

1.Set PTR:=START.

2.REPEAT STEP 3 WHILE(PTR!=NULL)

3.IF ITEM=INFO[PTR],THEN: SET LOC:=PTR AND EXIT.

ELSE: PTR:=LINK[PTR] [PTR now point to the next node]

[END IF]

[END of step 2LOOP]

4.SET LOC:=NULL [searching is unsuccessful]

5.EXIT

Memory Allocation & Garbage Collection in Linked Lists

- When working with **linked lists**, memory management is critical. The system must allocate and free memory efficiently.

Free Storage List (Free Pool)

- A **special list** that maintains **unused memory cells**.
- When a node is **deleted**, it is **added** to this **free list** for future use.
- This prevents **memory fragmentation** and reduces the number of calls to malloc().

How the Free List Works

1. **When a node is deleted**, it is added to the **free list** instead of being destroyed.
2. **When a new node is needed**, it is **taken** from the **free list** (if available).
3. If the **free list is empty**, a **new node is allocated using malloc()**.

Garbage Collection

Garbage collection is a process used to reclaim unused memory **automatically**.

Steps of Garbage Collection

1. **Mark Phase**: The computer **tags memory cells** that are currently **in use**.
2. **Sweep Phase**: It **collects all untagged (unused) memory** and adds it to the **free storage list**.

Overflow and Underflow

Overflow (Memory Exhaustion)

- Happens when **new data needs to be inserted**, but there is **no available memory**.
- Occurs when
  - The **free list is empty** and there is **no free memory left**.
  - malloc() **fails to allocate memory**.

Underflow (Empty Structure)

- Happens when **trying to delete an item** from an **empty** list.
- Occurs when:
  - The linked list is **already empty**, and deletion is attempted.

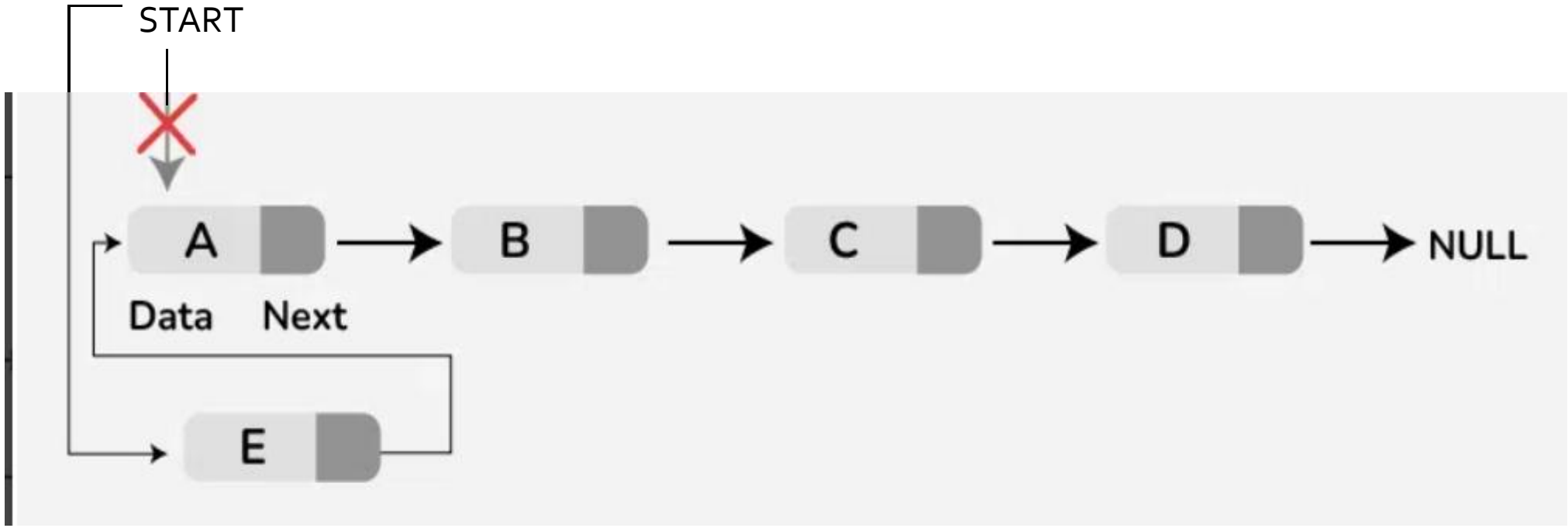
Inserting a New Node in a Linked List

- In this section, we will see how a new node is added into an already existing linked list.
- We will take four cases and then see how insertion is done in each case.
- **Case 1**: The new node is inserted at the beginning.
- **Case 2:** The new node is inserted at the end.
- **Case 3:** The new node is inserted after a given node.
- **Case 4**: The new node is inserted before a given node.

Inserting a Node at the Beginning of a Linked List.

ALGORITHM:

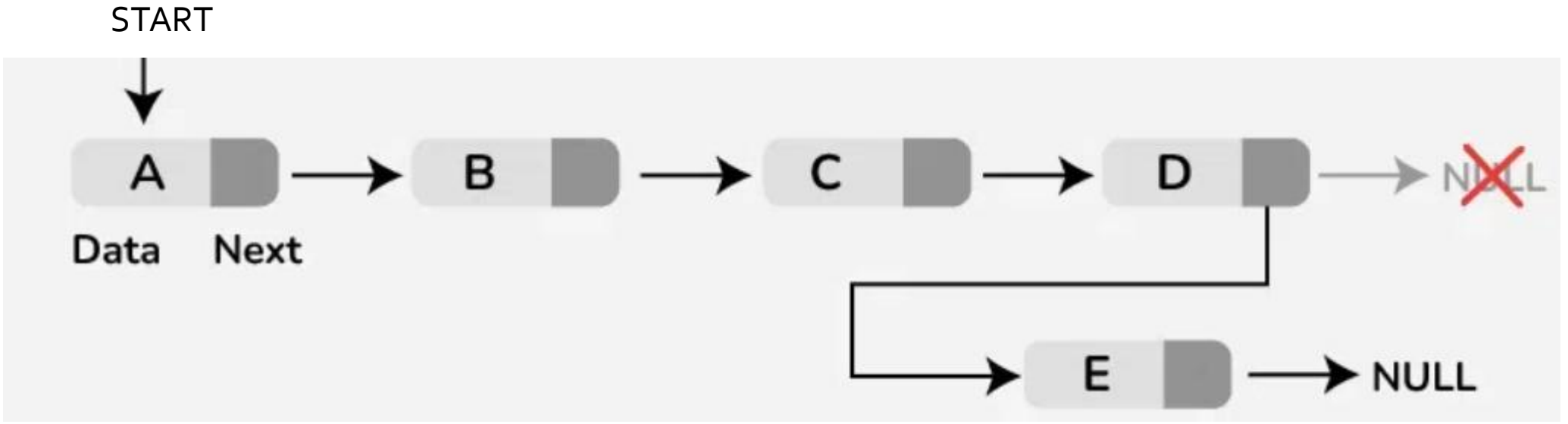
INSFIRST(INFO,LINK,START,AVAIL,ITEM)
This algorithm insert ITEM as the first node in the list.
1.[OVERFLOW?]if AVAIL=NULL,then:
Write:OVRFLOW, and Exit.
2.[REMOVE FIRST NODE FROM AVAIL list]
Set NEW:=AVAIL and AVAIL:=LINK[AVAIL].
3.set INFO[NEW]:=ITEM[Copies new data into new node].
4.Set LINK[NEW]:=START.[New node now points to orginal first node]
5.Set START:=NEW [Changes START so it points to the new node.]
6.Exit.


START
A
B
C
D
NULL
Data
Next
E

Inserting a Node at the End of a Linked List.

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 10
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET NEW_NODE->DATA = VAL
Step 5: SET NEW_NODE->NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR->NEXT != NULL
Step 8:      SET PTR = PTR->NEXT
        [END OF LOOP]
Step 9: SET PTR->NEXT = NEW_NODE
Step 10: EXIT
```

Figure 6.15 Algorithm to insert a new node at the end

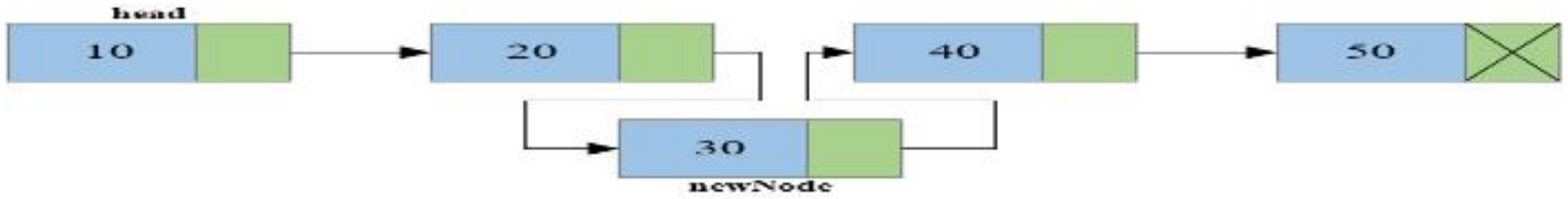

START
A
B
C
D
NULL
Data
Next
E
NULL

Insert an item after a given node of a linked list

- **Steps**
- Check if memory (AVAIL) is available. If not, print "OVERFLOW" and exit.
- Remove a node from AVAIL to create a new node (NEW).
- Store ITEM in INFO[NEW].
- If LOC == NULL, insert at the **beginning** by updating START.
- Else, insert **after LOC** by updating links.
- Exit.

ALGORITHM

INSLOC(INFO,LINK,START,AVAIL,LOC,ITEM)
1.IF(AVAIL=NULL),THEN WRITE:OVERFLOW AND EXIT.
2.[REMOVE A NODE FROM AVAIL]
NEW:=AVAIL AND AVAIL:=LINK[AVAIL]
3.SET INFO[NEW]:=ITEM
4.IF LOC=NULL,THEN: [FIRST LOCTION]
SET LINK[NEW]:=START AND START:=NEW.
5. ELSE:[INSERT A NODE WITH LOC.]
SET LINK[NEW]:=LINK[LOC] AND LINK[LOC]:=NEW.
[END IF]
6.EXIT



Delete the First Node from a Linked List

- **Steps to Delete the First Node from a Linked List**
- **Step 1: Check if the List is Empty**
- If START = NULL, print **"UNDERFLOW"** and exit (this means the list is already empty).
- **Step 2: Identify the Node to Delete**
- Set LOC = START (store the location of the first node).
- **Step 3: Update the START Pointer**
- Set START = LINK[START] (move START to the second node).
- **Step 4: Return the Deleted Node to the AVAIL List**
- Set LINK[LOC] = AVAIL (link the deleted node to the available list).
- Set AVAIL = LOC (update AVAIL to point to the deleted node).
- **Step 5: Exit the Algorithm**

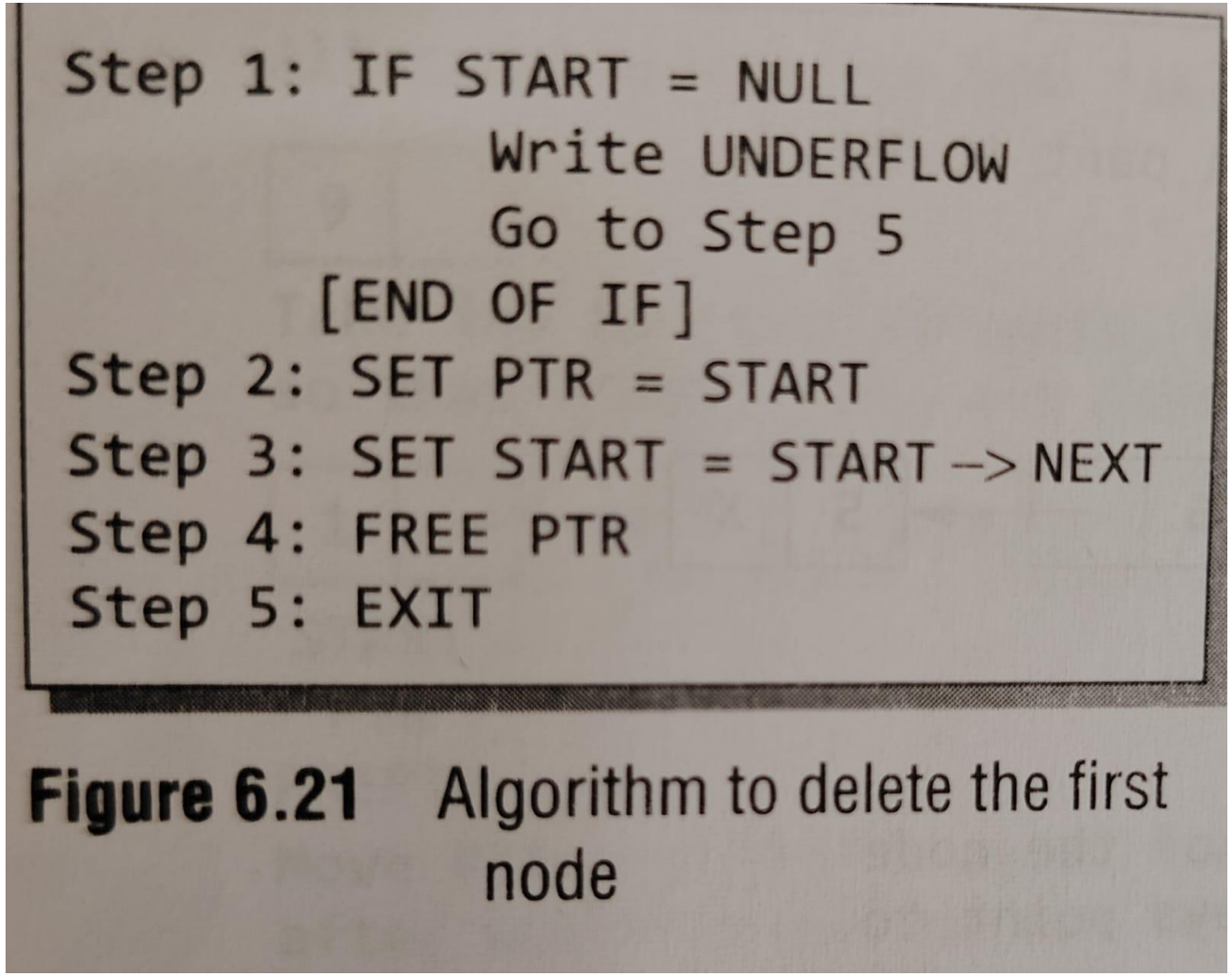


Figure 6.21 Algorithm to delete the first node

Delete a node After a given node

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 10
        [END OF IF]
Step 2: SET PTR = START
Step 3: SET PREPTR = PTR
Step 4: Repeat Steps 5 and 6 while PREPTR -> DATA != NUM
Step 5:      SET PREPTR = PTR
Step 6:      SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 7: SET TEMP = PTR
Step 8: SET PREPTR -> NEXT = PTR -> NEXT
Step 9: FREE TEMP
Step 10: EXIT
```

Figure 6.25 Algorithm to delete the node after a given node

Header Linked List

- A header linked list is a type of linked list that has a header node at the beginning of the list. In a header linked list, HEAD points to the header node instead of the first node of the list.
- The header node does not represent an item in the linked list. This data part of this node is generally used to hold any global information about the entire linked list. The next part of the header node points to the first node in the list.

A header linked list can be divided into two types:

- Grounded header linked list that stores NULL in the last node's next field.
- Circular header linked list that stores the address of the header node in the next part of the last node of the list